MID-STATE TECHNICAL COLLEGE · IT PROGRAMS

Processes and Monitoring

Unit 8 · Linux (10-151-105)

MID-STATE TECHNICAL COLLEGE

Learning Outcomes

By the end of this unit, students will be able to:

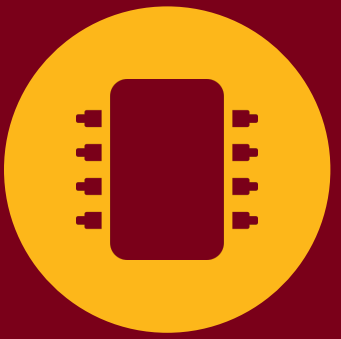
- **Describe a process —** and identify it by its process ID (PID).
- **Recognize process states —** and what each one signals.
- **View and monitor —** processes and system resources — CPU, memory, and load.
- **Send signals —** to control and terminate processes.
- **Adjust priority —** and manage foreground and background jobs.
- **Diagnose a runaway process —** and recognize abnormal activity.

IN THIS UNIT

1. **Understanding Processes**
2. **Viewing and Monitoring**
3. **Controlling Processes**
4. **Investigating and Troubleshooting**

LESSON 1

Understanding Processes



What Is a Process?

- **Program —** a file on disk — inert, doing nothing on its own.
- **Process —** a running instance of a program, loaded into memory and executing.
- **Many at once —** the same program can run several times; each instance is its own process.
- **Started by a parent —** every process records a parent process ID (PPID), so you can trace its origin.

KEY TERMS

`PID` process identifier — a unique number the system uses to track each process.

`PPID` parent process ID — the process that started this one.

Linux tracks processes by their unique PID rather than by name, because several processes can share a name but a PID is always unique.

How Processes Are Created

A process can come into being in a few different ways:

Compiled programs

Software written in a language like C and compiled to a binary. Running it creates a process.

Shell built-ins

Commands such as cd and exit are built into the shell itself, not separate files.

Shell scripts

Text files of commands the shell reads and runs line by line — interpreted, not compiled.

MULTITASKING

Even on a single CPU, Linux appears to run many processes at once by switching between them extremely quickly — one task at a time, but fast enough to look simultaneous. Systems with multiple cores genuinely run processes in parallel, and the kernel decides which process gets the processor next.

Process States

Monitoring tools label each process with a one-letter state code. Knowing them turns a wall of output into a quick diagnosis.

State	What it means
Running (R)	Executing on the CPU, or ready and waiting in the run queue for its turn.
Sleeping (S)	Waiting for input, a reply, or I/O. Interruptible — a signal can wake it. The normal idle state.
Uninterruptible Sleep (D)	Waiting on hardware (usually disk I/O) and cannot be interrupted. Long stays here suggest a storage problem.
Stopped (T)	Paused, typically after a stop signal such as Ctrl+Z. Stays frozen until told to continue.
Zombie (Z)	Finished, but its table entry lingers until the parent collects its exit status. A pile signals a bad parent.
Idle (I)	A background or kernel task that is ready but not currently doing work.

LESSON 2

Viewing and Monitoring

Viewing Processes with ps

The classic tool for a point-in-time snapshot of processes is ps. Its power comes from options that widen and shape the view:

- **ps -ef —** every process in full detail — owner, PID, PPID, and full command.
- **ps aux —** a complete listing that also reports CPU and memory percentage per process.
- **ps -o pid,ppid,cmd —** a custom view of exactly the columns you name.

HEADS-UP

ps accepts three option styles — Unix (ps -e), GNU (ps --help), and BSD (ps aux). They are not interchangeable, and mixing them can give surprising results. With so many options, ps is the textbook case for keeping man ps close at hand.

Finding and Relating Processes

- **pgrep —** finds processes whose name contains a string. Add -l to print the name with the PID.
- **pidof —** similar, but expects the exact process name — more precise, less forgiving.
- **pstree —** draws processes as a branching tree, showing which process spawned which.

```
$ pgrep -l ssh          # names + PIDs matching "ssh"
$ pidof sshd            # PID of the exact process
$ pstree                # the full process hierarchy
```

PROCESS HIERARCHY

```
systemd
  ├─ NetworkManager
  ├─ cron
  ├─ sshd ── sshd ── bash
  └─ nginx ── nginx
```

The first system process sits at the root, with services and shells branching beneath it.

Live Monitoring: top

Where ps gives a still photograph, top gives a live feed that refreshes every few seconds — one of the first commands an administrator reaches for.

- **Summary header —** uptime, load average, memory use, and a process count at a glance.
- **Ranked list —** processes sorted by how much they are consuming, updated live.
- **Interactive —** respond to keypresses while it runs.

INTERACTIVE KEYS

`Shift+P` sort by CPU usage
`Shift+M` sort by memory usage
`q` quit

Reading the CPU Line in top

Each figure on top's CPU summary line tells you something different about why the processor is busy:

Metric	What it tells you
`us (user)`	Time spent running normal user programs.
`sy (system)`	Time spent running the kernel on behalf of those programs.
`id (idle)`	Time the CPU did nothing. A high number is good — it means headroom.
`wa (I/O wait)`	CPU ready, but blocked waiting on disk or other I/O. Persistently high points at a storage bottleneck.
`st (steal)`	On a VM, time this CPU waited for the physical host's CPU. High steal means the host is oversubscribed.

htop and atop

`htop`

A friendlier, color-coded version of top with bar graphs, easier scrolling, and simple process selection. Many administrators install it as their default.

`atop`

Goes further by logging performance over time across CPU, memory, disk, and network — valuable for spotting trends and reviewing what happened earlier, not just now.

DUAL-DISTRO NOTE

top, ps, uptime, free, and vmstat ship on both Rocky and Ubuntu. The extras — htop and atop — usually are not installed by default. Add them with the system's package manager: "sudo dnf install htop" on Rocky, "sudo apt install htop" on Ubuntu. The commands behave identically once installed; only the way you add them differs.

CPU and Load

- **uptime —** shows how long the system has run and the load average over the last 1, 5, and 15 minutes.
- **Load average —** counts processes running or waiting to run. Compare it against the number of CPU cores.
- **Above the core count —** more work is queuing than the system can keep up with.
- **mpstat / pidstat / sar —** deeper CPU reports (per core, per process, over time) from the sysstat package.

READ IT RIGHT

High CPU usage means a process is maxing the processor. But high I/O wait (wa) means the CPU is healthy and starved by slow storage, and a high load average relative to your cores means work is piling up. The same number can have very different causes — read the metric, not just the alarm.

Memory and the OOM Killer

- **free -h —** reports used and available memory in human-readable units.
- **vmstat —** reports virtual-memory activity — swapping between RAM and disk. Heavy use is slow; you want it low.
- **watch —** re-runs a command at intervals; "watch -d free -h" highlights what changed.

```
$ free -h             # memory used vs available
$ vmstat 1            # virtual-memory activity, live
$ watch -d free -h    # refresh and highlight changes
```

OUT OF MEMORY

A memory leak is a program that requests memory and never releases it. To survive, the kernel's Out-of-Memory (OOM) Killer terminates a process to reclaim memory — so an app can vanish without warning. It records what it killed in the log: on Rocky that is /var/log/messages; on Ubuntu, check the journal with journalctl.

LESSON 3

Controlling Processes

Process Signals

When a process will not end on its own, you intervene by sending it a signal. Three do most of the work, from polite to forceful:

Signal	What it does
`SIGTERM (15)`	Politely asks the process to shut down cleanly — finish writing files, release resources, exit. The default; try it first.
`SIGKILL (9)`	Forces immediate termination with no cleanup. Data loss is possible. A last resort, used only when SIGTERM fails.
`SIGHUP (1)`	Originally "hang up" — signals that the controlling terminal closed; often used to ask a service to reload its configuration.

Try SIGTERM first; save SIGKILL for when nothing else works.

Ending Processes: kill, killall, pkill

- **kill {signal} {PID} —** targets one process by PID.
- **killall {name} —** targets every process with a given exact name at once.
- **pkill {pattern} —** matches by pattern, and can select by user or other criteria.
- **sudo —** is needed to signal processes owned by other users or the system.

```
$ sudo kill -15 1312      # ask PID 1312 to exit
$ sudo kill -9 1917       # force PID 1917 to stop
$ sudo killall firefox    # all "firefox" processes
$ sudo pkill -15 firefox  # SIGTERM by pattern
```

GENTLER FIRST

Before reaching for kill, remember the softer options: a GUI app's close button, systemctl stop for a service (Unit 9), or Ctrl+C to interrupt a command in your terminal. Save kill -9 for when nothing else works.

Controlling Priority: nice and renice

Niceness runs from -20 (highest priority, most CPU) to 19 (lowest); 0 is the default. A higher number is "nicer" — it yields CPU to others.

- **nice -n {value} {command} —** starts a new process at a chosen niceness.
- **renice {value} -p {PID} —** changes the niceness of a process already running.

```
$ sudo nice -n -10 ./backup.sh   # start higher
$ sudo renice -13 -p 1218        # raise a running job
```

PRIVILEGE NOTE

A regular user can only make their own processes nicer (raise the value toward 19) — they cannot claim more CPU at others' expense. Lowering niceness into the negative range, or adjusting another user's process, requires sudo or root. This keeps any one user from hogging a shared system's processor.

Job Control: Foreground and Background

Jobs are the processes you start in your own shell. Job control lets you juggle several without tying up your terminal.

- **command &** — start a command in the background straight away.
- **Ctrl+Z then bg** — pause a foreground job, then resume it in the background.
- **jobs** — list your jobs, each with a number in brackets.
- **fg %n / bg %n** — move job n to the foreground or background.
- **nohup command &** — keep a job running after you log out.

```
$ backup.sh &          # run in background
  ^Z                   # pause foreground job
$ bg                   # resume in background
$ jobs                 # list jobs
$ fg %1                # bring job 1 forward
$ nohup backup.sh &    # survive logout
```

LESSON 4

Investigating and Troubleshooting

Looking Deeper: /proc and strace

When standard commands aren't enough, Linux exposes per-process detail through the /proc virtual filesystem — generated by the kernel on the fly, one directory per PID.

- **/proc/{PID}/cmdline —** the full command and arguments that launched it.
- **/proc/{PID}/exe —** a link to the actual executable on disk.
- **/proc/{PID}/status —** runtime detail, including memory use (VmRSS).

STRACE

strace traces the system calls a process makes — its requests to the kernel to read files, allocate memory, and so on. Trace a command as you launch it (strace ls) or attach to a running one (strace -p {PID}). When a process is mysteriously stuck, the last call it is waiting on is often the clue.

Troubleshooting a Runaway Process

When a system slows to a crawl, the method is always the same:

1. **Measure** Open top (or htop) and sort by CPU or memory. Check uptime for load average and free -h for memory pressure.

2. **Identify** Note the offending PID, its owner, and its command. Confirm with ps -ef | grep, and check /proc/{PID}/exe if it looks suspicious.

3. **Decide** Is it legitimate-but-greedy (renice it) or does it need to stop? Is the bottleneck really CPU, or is it I/O wait pointing at disk?

4. **Act** Lower its priority or end it — kill -15 first, kill -9 only if it refuses. Then re-measure to confirm the system recovered.

Security Focus: Process Visibility

Monitoring as a Detection Skill

- An attacker's foothold almost always shows up as a process — a miner pegging the CPU, an unfamiliar program in a temp directory, a script run by the wrong user.
- When CPU or memory is unexpectedly pinned, identify the exact process and who owns it (top, ps -ef, pgrep -l).
- Treat resource anomalies as a signal: sustained high CPU on an idle server is a classic sign of cryptomining malware.
- Know what normal looks like — a performance baseline is what lets you recognize the abnormal.
- Before you kill a suspicious process, capture what it is — its command line, working directory, and open files — so you can investigate after.

Why it matters: you can't defend a system you aren't watching. This monitoring fluency is the foundation of detection — connected to logs and access control in Unit 11.

Lab: Processes and Monitoring

You'll perform these tasks on both Rocky Linux 10 and Ubuntu Server 24.04, starting from the WK08_PROCESSES snapshot.

- **Inspect —** list and filter running processes and identify their owners and states.
- **Monitor —** read live CPU, memory, and load metrics and interpret what they mean.
- **Signal —** terminate processes gracefully and forcefully with kill, killall, and pkill.
- **Prioritize —** adjust process priority with nice and renice.

SCENARIO

The lab ends with a runaway-process scenario you diagnose and resolve on your own — no step-by-step walkthrough — the way you would on a real server. See the lab document for scheduling and submission.

Key Takeaways

- **Everything running is a process —** tracked by a unique PID, started by a parent (PPID).
- **State tells the story —** R and S are normal; D and a pile of Z are worth investigating.
- **ps snapshots, top watches —** and htop / atop extend the live view.
- **Read the metric, not the alarm —** high CPU, high I/O wait, and high load mean different things.
- **Stop gently, then firmly —** SIGTERM first, SIGKILL last; renice instead of killing when you can.
- **Monitoring is detection —** abnormal resource use is often the first sign of compromise.

MID-STATE TECHNICAL COLLEGE · IT PROGRAMS

Questions?

Unit 8 · Linux (10-151-105)

MID-STATE TECHNICAL COLLEGE